

# Data Analyse Agent

## 综合测试卷（二）：Agent 执行链路与工具协作

ScopeRouter、Planner、Tool Policy、SSE、前端计划面板与工具调用

|      |                                                 |
|------|-------------------------------------------------|
| 项目名称 | Data Analyse Agent                              |
| 总分   | 100 分                                           |
| 题型   | 填空题 40 分；简答题与分析题 60 分                           |
| 考查重点 | Plan-and-Execute、工具可见性控制、SSE 事件、工具结果、前端状态与多轮上下文 |
| 版本说明 | 依据当前项目代码、README、evals、tests 与实现说明生成             |

使用说明：本 PDF 为“题目 + 标准答案 + 详细解析”的完整复习版。每题均标注项目知识点或代码位置，便于回到源码中复盘。

# 目录与分值校验

一、填空题：20 道，每题 2 分，共 40 分。

二、简答题与分析题：10 道，每题 6 分，共 60 分。

总分校验：40 + 60 = 100 分。

| 部分      | 题量 | 单题分值 | 小计    |
|---------|----|------|-------|
| 填空题     | 20 | 2 分  | 40 分  |
| 简答题与分析题 | 10 | 6 分  | 60 分  |
| 合计      | 30 | -    | 100 分 |

覆盖范围：项目背景、目录结构、数据集持久化、Manifest、SQLite、关系配置、API、ScopeRouter、Planner、工具策略、Python 沙箱、Artifact Store、上下文压缩、前端 SSE 展示、自动化评测与优化方向。

## 第一部分：填空题（共 40 分）

1. 题目（2 分）：浏览器默认使用的流式 Agent 主链路函数是 \_\_\_\_\_。

标准答案

stream\_data\_agent\_events

详细解析

chat\_stream 调用 stream\_data\_agent\_events，并将其产生的事件格式化为 SSE。该函数实现 ScopeRouter、Planner、Step Executor、Synthesizer 的流程。

对应项目知识点或代码位置：app/api/chat.py；app/agent/runtime.py。

2. 题目（2 分）：在流式链路中，进入 Planner 前首先由 \_\_\_\_\_ 判断问题是否适合当前数据集。

标准答案

ScopeRouter

详细解析

runtime.py 先 yield thinking，然后调用 classify\_scope。只有 should\_plan 为 true 才继续 build\_execution\_plan。

对应项目知识点或代码位置：app/agent/runtime.py；app/agent/scope\_router.py。

3. 题目（2 分）：ScopeRouter 支持的四种 scope 是 in\_scope、out\_of\_scope、needs\_clarification 和 \_\_\_\_\_。

标准答案

general\_help

详细解析

scope\_router.py 的 Literal 定义和 README 表格都列出这四类。general\_help 用于系统用法、工具原理、Docker 等问题。

对应项目知识点或代码位置：app/agent/scope\_router.py；README.md：ScopeRouter。

4. 题目（2 分）：Planner 输出的执行计划模型类是 \_\_\_\_\_。

标准答案

ExecutionPlan

详细解析

planner.py 定义 ExecutionPlan，包含 plan\_id、mode、primary\_intent、user\_goal、steps 和 final\_response\_requirements。

对应项目知识点或代码位置：app/agent/planner.py。

5. 题目（2 分）：每个计划步骤的模型类是 \_\_\_\_\_，其中包含 intent、goal、allowed\_tools、preferred\_tool、depends\_on 等字段。

标准答案

PlanStep

详细解析

PlanStep 在 model\_validator 中会根据 intent 清洗 allowed\_tools，并计算 retry\_limit。

对应项目知识点或代码位置：app/agent/planner.py。

6. 题目（2分）：第一版 Planner 最多支持 \_\_\_\_\_ 个步骤。

标准答案

5

详细解析

tool\_policy.py 中 MAX\_PLAN\_STEPS = 5，ExecutionPlan 的 validator 会截断 steps。

对应项目知识点或代码位置：app/agent/tool\_policy.py；app/agent/planner.py。

7. 题目（2分）：控制 intent 到工具可见性映射的字典是 \_\_\_\_\_。

标准答案

INTENT\_TOOL\_POLICY

详细解析

tool\_policy.py 中定义 query、chart、quality、advanced、cleaning、mixed 对应工具集合，运行时每 step 只绑定 allowed\_tools。

对应项目知识点或代码位置：app/agent/tool\_policy.py。

8. 题目（2分）：intent 为 advanced 时，允许的工具是 \_\_\_\_\_。

标准答案

python\_analysis

详细解析

README 和 tool\_policy.py 均说明 advanced 只允许 python\_analysis，用于聚类、建模、时间序列、复杂统计等。

对应项目知识点或代码位置：README.md；app/agent/tool\_policy.py。

9. 题目（2分）：intent 为 chart 时，允许的工具是 \_\_\_\_\_。

标准答案

generate\_chart

详细解析

chart 意图只允许 generate\_chart，避免基础图表问题被错误路由到 Python。

对应项目知识点或代码位置：app/agent/tool\_policy.py。

10. 题目（2 分）：清洗类工具默认重试次数是 \_\_\_\_\_。

标准答案

0

详细解析

TOOL\_RETRY\_LIMITS 中 suggest\_cleaning、apply\_cleaning、reset\_cleaning 都是 0。清洗涉及数据修改或状态变化，失败不自动反复尝试。

对应项目知识点或代码位置：app/agent/tool\_policy.py。

11. 题目（2 分）：python\_analysis 的失败重试次数是 \_\_\_\_\_。

标准答案

3

详细解析

TOOL\_RETRY\_LIMITS 中 python\_analysis 为 3，符合之前设计：Python 代码生成/修复比 SQL 更需要重试。

对应项目知识点或代码位置：app/agent/tool\_policy.py。

12. 题目（2 分）：当前步骤绑定工具时，runtime 调用模型的 \_\_\_\_\_ 方法，只向模型暴露 selected\_tools。

标准答案

bind\_tools

详细解析

stream\_data\_agent\_events 中 model\_with\_tools = reason\_model.bind\_tools(selected\_tools)，是控制工具可见性的关键。

对应项目知识点或代码位置：app/agent/runtime.py。

13. 题目（2 分）：每个 step 内最多工具调用轮数由常量 \_\_\_\_\_ 控制，当前值为 6。

标准答案

MAX\_STEP\_TOOL\_ROUNDS

详细解析

runtime.py 中定义 MAX\_STEP\_TOOL\_ROUNDS = 6，用于防止某个步骤陷入无限工具循环。

对应项目知识点或代码位置：app/agent/runtime.py。

14. 题目（2 分）：当模型只返回 tool\_calls 而没有正文解释时，系统会通过 \_\_\_\_\_ 生成工具调用原因。

标准答案

\_generate\_tool\_reason

### 详细解析

`_execute_plan_step` 检查 `response.content` 为空时会 `yield tool_reason`，并调用 `_generate_tool_reason`。该函数隐藏 `dataset_id`、`max_rows` 等参数。

对应项目知识点或代码位置：`app/agent/runtime.py`。

15. 题目（2 分）：判断工具结果是否成功的函数是 \_\_\_\_\_，它优先读取 JSON 中的 `ok` 字段。

### 标准答案

`_tool_result_success`

### 详细解析

`_tool_result_success` 会解析 JSON；若存在 `ok` 字段则按 `bool(ok)` 判定，比单纯关键词判断更可靠。

对应项目知识点或代码位置：`app/agent/runtime.py`。

16. 题目（2 分）：工具执行结束后，`runtime` 会调用 \_\_\_\_\_ 把工具结果转换为 Artifact。

### 标准答案

`build_tool_artifacts`

### 详细解析

`_execute_plan_step` 在 `yield tool_end` 后调用 `build_tool_artifacts`，并继续 `yield artifact` 事件。

对应项目知识点或代码位置：`app/agent/runtime.py`；`app/agent/artifacts.py`。

17. 题目（2 分）：从 Python 或图表工具结果中提取图表 SSE 事件的函数是 \_\_\_\_\_。

### 标准答案

`_try_build_chart_events`

### 详细解析

该函数解析 `tool_result` JSON，收集 `payload.figures`、`result.figures` 或 `chart_url/chart_id`，并去重后 `yield chart` 事件。

对应项目知识点或代码位置：`app/agent/runtime.py`。

18. 题目（2 分）：`query_data` 工具成功时返回 DataFrame 的 \_\_\_\_\_ 格式文本。

### 标准答案

Markdown 表格

### 详细解析

`make_query_data_tool` 中 `return result.to_markdown(index=False)`。这也是 Artifact 中估算 SQL 表格预览行数的依据。

对应项目知识点或代码位置：`app/agent/tools.py`；`app/agent/artifacts.py`。

19. 题目（2 分）：前端输入框上方用于展示执行计划的折叠面板元素 id 是 \_\_\_\_\_。

标准答案

plan-console

详细解析

index.html 中新增 details#plan-console，app.js 通过 showPlanConsole/updatePlanStepStatus 更新状态。

对应项目知识点或代码位置：frontend/index.html；frontend/app.js。

20. 题目（2 分）：SSE 事件中，用于表示步骤开始和结束的事件分别是 \_\_\_\_\_ 和 \_\_\_\_\_。

标准答案

plan\_step\_start；plan\_step\_end

详细解析

runtime.py 在每个步骤执行前后 yield 两类事件，chat.py 保存为 role=plan\_step，前端据此还原状态。

对应项目知识点或代码位置：app/agent/runtime.py；app/api/chat.py；frontend/app.js。

## 第二部分：简答题与分析题（共 60 分）

1. 题目（6 分）：完整梳理一次 `/api/chat/stream` 请求从进入 API 到前端收到最终回答的调用链。

### 标准答案

参考答案：chat\_stream 解析 conversation/dataset，必要时压缩上下文，保存用户消息，发送 context 和 conversation 事件；调用 stream\_data\_agent\_events；runtime 获取 schema 和 tools，先 ScopeRouter，再 Planner；按 step 绑定 allowed\_tools，调用工具并发出 tool\_start/tool\_end/artifact/chart；最后 Synthesizer 汇总 text\_delta，API 保存 assistant 文本和事件，前端按 SSE 更新聊天区和 plan-console。

### 评分标准

- API 解析和上下文 1 分
- ScopeRouter 1 分
- Planner 和 step executor 1.2 分
- 工具事件与 artifact/chart 1.3 分
- Synthesizer 和保存历史 1 分
- 前端 SSE 展示 0.5 分

### 详细解析

这条链路横跨 API、ContextService、DatasetService、Agent runtime、工具层、Artifact Store 和前端。答题应体现事件流而不是只说“模型回答”。容易漏掉的是 scope 不通过时不会进入 plan，也不会调用工具。

对应项目知识点或代码位置：app/api/chat.py；app/agent/runtime.py；frontend/app.js。

2. 题目（6 分）：解释“每个 step 只暴露 allowed\_tools”如何实现，以及它解决了什么问题。

### 标准答案

参考答案：Planner 为每个 PlanStep 生成 intent 和 allowed\_tools，PlanStep validator 用 tool\_policy 清洗工具列表。runtime 每步构造 selected\_tools 后调用 reason\_model.bind\_tools(selected\_tools)，所以模型只能调用当前 step 允许的工具。它解决“所有问题都走 SQL”或“复杂问题误用简单工具”的问题，也降低误调用清洗/危险工具的风险。

### 评分标准

- 说明 Planner/PlanStep 产生 allowed\_tools 1.5 分
- 说明 tool\_policy 清洗 1 分
- 说明 runtime bind\_tools 1.5 分
- 说明解决工具误路由 1.5 分
- 说明安全收益 0.5 分

### 详细解析

核心不是提示词一句话，而是运行时重新 bind\_tools 的硬约束。allowed\_tools 来自 INTENT\_TOOL\_POLICY，preferred\_tool 不在 allowed\_tools 时会被纠正。



对应项目知识点或代码位置：app/agent/planner.py；app/agent/tool\_policy.py；app/agent/runtime.py。

3. 题目（6分）：比较 query\_data、generate\_chart 与 python\_analysis 三个工具的数据流差异。

#### 标准答案

参考答案：query\_data 直接用 SQLService.query 执行只读 SQL 并返回 Markdown 表格；generate\_chart 先 SQLService.query 得到 DataFrame，再 ChartService 生成 PNG 并返回 chart\_url；python\_analysis 先 SQLService.query\_for\_analysis 导出更多行，再 PythonSandboxService 写入 data.json/schema.json/analysis.py，Docker 执行后读取 result.json 并发布 figures。

#### 评分标准

- query\_data 流程 1.5 分
- generate\_chart 流程 1.5 分
- python\_analysis 流程 2 分
- 指出共同依赖 SQLService 0.5 分
- 指出适用场景差异 0.5 分

#### 详细解析

三者都依赖 SQLService，但后续处理不同。chart 是基础可视化，python\_analysis 是复杂分析 fallback。max\_rows 也不同，query\_data 默认展示 100/上限 1000，python 分析可导出更多行。

对应项目知识点或代码位置：app/agent/tools.py；app/services/sql\_service.py；app/services/chart\_service.py；app/services/python\_sandbox\_service.py。

4. 题目（6分）：分析 ScopeRouter 为什么要放在 Planner 之前，而不是让 Planner 自己判断所有问题。

#### 标准答案

参考答案：ScopeRouter 先判断问题是否和当前数据集相关，能拦截 out\_of\_scope、general\_help、needs\_clarification，避免 Planner 为股市、天气、系统用法等问题牵强生成 SQL/Python 计划。这样减少工具误调用、降低成本，并让用户得到更诚实的限制说明。

#### 评分标准

- 说明拦截范围 1.5 分
- 说明避免错误 Planner 1.5 分
- 说明成本和安全收益 1 分
- 结合股市例子 1 分
- 指出第一版局限 1 分

#### 详细解析

之前“明年股市会好起来没啊”被错误规划成电商销售趋势分析，就是缺少前置 scope 的典型问题。ScopeRouter 的 rule 层和 LLM JSON fallback 共同工作，但第一版仍依赖关键词和 schema 线索。

对应项目知识点或代码位置：app/agent/scope\_router.py；app/agent/runtime.py；README.md：ScopeRouter。

5. 题目（6 分）：说明 Planner 的 fallback 机制如何工作，以及它的价值和局限。

#### 标准答案

参考答案：build\_execution\_plan 先尝试让模型输出 JSON 并校验为 ExecutionPlan；若校验失败、解析失败或异常，就调用 build\_fallback\_plan。fallback 通过关键词检测 intent，如画图、聚类、清洗、统计等，构造最多 5 个 PlanStep。价值是 LLM 不稳定时仍能运行；局限是关键词粗糙，复杂语义和字段依赖不如模型规划。

#### 评分标准

- 说明模型 JSON 路径 1 分
- 说明异常时 fallback 1 分
- 说明关键词检测 intent 1.5 分
- 说明价值 1 分
- 说明局限 1 分
- 提到清洗安全 0.5 分

#### 详细解析

fallback 是工程韧性设计。它不能替代高质量 Planner，但能保证基本问题不因 JSON 格式错误直接失败。清洗 intent 还会经过 \_enforce\_cleaning\_safety。

对应项目知识点或代码位置：app/agent/planner.py；tests/test\_planner\_policy.py。

6. 题目（6 分）：解释工具调用失败后的重试策略如何体现不同工具风险。

#### 标准答案

参考答案：retry\_limit\_for\_tools 从 TOOL\_RETRY\_LIMITS 取最大值：python\_analysis 为 3，query\_data 和 generate\_chart 为 2，清洗和固定分析为 0。Python 代码生成容易有语法/路径/result.json 问题，因此允许更多修复；SQL/图表可有限修正；清洗涉及状态变更，不应自动反复执行。

#### 评分标准

- 列出 retry 数字 2 分
- 说明 Python 为什么 3 1 分
- 说明 SQL/图表为什么 2 1 分
- 说明清洗为什么 0 1 分
- 说明 runtime 超限处理 1 分

#### 详细解析

runtime 中失败会增加 failed\_attempts，超过 step.retry\_limit 后追加 HumanMessage 告诉模型停止重试。工具风险越高，越需要谨慎；清洗 0 次反映了数据修改操作的保守策略。

对应项目知识点或代码位置：app/agent/tool\_policy.py；app/agent/runtime.py。

7. 题目（6 分）：分析 tool\_reason 事件的作用，以及为什么要隐藏部分参数。

#### 标准答案

参考答案：tool\_reason 在工具调用前向用户解释本轮操作原因，提升可解释性。若模型只输出结构化 tool\_calls 没有自然语言，runtime 调用 \_generate\_tool\_reason。它构造 visible\_args 时排除 dataset\_id、max\_rows，避免泄露不必要内部标识和噪声参数，同时保留 SQL、analysis\_goal 等用户能理解的信息。

#### 评分标准

- 说明解释工具原因 1.5 分
- 说明触发条件 1 分
- 说明隐藏 dataset\_id/max\_rows 1 分
- 说明用户体验/审计价值 1 分
- 说明 fallback 1.5 分

#### 详细解析

这不是核心计算逻辑，而是交互体验和可审计性设计。它也在 prompt 中要求不要泛泛复述工具名称，若 LLM 调用失败则回退到 \_build\_tool\_reason。

对应项目知识点或代码位置：app/agent/runtime.py：\_generate\_tool\_reason、\_build\_tool\_reason。

8. 题目（6 分）：前端 plan-console 如何与后端 SSE 事件协作显示步骤状态？

#### 标准答案

参考答案：后端 runtime 发出 plan、plan\_step\_start、plan\_step\_end；chat.py 将 plan\_step 事件保存进对话。前端 app.js 收到 plan 时 showPlanConsole，收到 start 时 updatePlanStepStatus 为 running，收到 end 时根据 success 设为 done 或 failed。index.html 中 details#plan-console 位于输入框上方，可折叠展开。

#### 评分标准

- 后端事件 1.5 分
- API 保存 plan\_step 1 分
- 前端函数和状态 2 分
- 可折叠面板位置 0.75 分
- 历史恢复 0.75 分

#### 详细解析

plan-console

不是单纯历史消息，它承担当前执行状态面板的职责。恢复历史时，renderConversationMessages 遇到 role=plan 和 role=plan\_step 也会重建状态。

对应项目知识点或代码位置：frontend/index.html；frontend/app.js；app/api/chat.py；app/agent/runtime.py。

9. 题目（6 分）：说明同一轮最终总结和下一轮历史上下文在工具结果可见性上的差异。

#### 标准答案

参考答案：同一轮内，runtime 会把完整 ToolMessage content 加入 messages，因此最终 Synthesizer 能看到当前轮完整工具结果；下一轮构造历史时，ContextService 会把历史 tool 转成 result\_preview，并把 artifact 转成 summary + preview，不再注入完整 result.json、长 stdout/stderr

或完整 `python_code`。

#### 评分标准

- 同轮完整 ToolMessage 2 分
- 下一轮 `result_preview/artifact` 2 分
- 说明设计取舍 1.5 分
- 指出常见误解 0.5 分

#### 详细解析

这个差异平衡了回答质量和上下文成本。同轮需要完整材料保证总结准确；跨轮只需事实索引，防止上下文膨胀。容易混淆的是 `artifact` 并没有完全替代同轮 ToolMessage。

对应项目知识点或代码位置：`app/agent/runtime.py`；`app/services/context_service.py`；`app/agent/artifacts.py`。

10. 题目（6 分）：如果复杂分析问题被错误规划成 `query_data`，你会如何定位和修复？

#### 标准答案

参考答案：先查看 `scope` 是否 `in_scope`；再检查 `plan-console` 中 `primary_intent`、`steps` 和 `allowed_tools`；查看 `planner_eval/tool_routing_eval` 是否缺对应用例；检查 `planner.py` 的关键词、系统提示和 `tool_policy`；必要时加入评测用例，如“聚类、热力图、时间序列”必须 `advanced/python_analysis`；最后运行 `evals` 和 `pytest`。

#### 评分标准

- 定位 `scope/plan` 1 分
- 检查 `allowed_tools/tool_policy` 1.2 分
- 检查 `Planner/fallback` 1.2 分
- 补评测 1.2 分
- 运行回归 1 分
- 区分 `intent` 错和 `allowed_tools` 错 0.4 分

#### 详细解析

这个问题可能来自 `ScopeRouter`、`Planner`、`tool_policy` 或提示词。修复不应只改 `prompt`，还要补评测防回归。若是模型输出 `allowed_tools` 错，`PlanStep validator` 会清洗；若 `intent` 本身错，需要改善 `Planner` 或 `fallback`。

对应项目知识点或代码位置：`app/agent/planner.py`；`app/agent/tool_policy.py`；`evals/cases/tool_routing_eval.jsonl`；`evals/runners/run_evals.py`。